

SISTEMA ESPECIALISTA EM PROLOG: GUIA DE PIZZA

LUISA CAETANO ARAÚJO¹

¹ Graduando em Ciência da Computação, IFMG - Câmpus Formiga, lluisacaetanoaraujo@gmail.com.

1. INTRODUÇÃO

Escolher uma pizza pode parecer algo simples, mas, com tantas opções de sabores, ingredientes e combinações possíveis, essa decisão nem sempre é tão fácil. Pensando nisso, este trabalho apresenta o desenvolvimento de um Sistema Especialista em Prolog capaz de recomendar pizzas de forma personalizada, levando em consideração as preferências do usuário, possíveis restrições alimentares, o contexto em que a pizza será consumida e até sugestões de harmonização com bebidas.

A escolha do tema se deve justamente à popularidade da pizza e à sua enorme variedade, que permite organizar o conhecimento de forma estruturada e explorar diferentes relações entre ingredientes e categorias, como pizzas tradicionais, especiais e doces. Além disso, esse cenário é ideal para a aplicação de regras de inferência, permitindo que o sistema vá além de sugestões simples e ofereça recomendações mais inteligentes e contextualizadas.

O sistema também busca atender diferentes perfis de usuários, incluindo opções para vegetarianos, sugestões adequadas para crianças, recomendações para ocasiões como festas e alternativas voltadas a diferentes tipos de dieta. Ao todo, o projeto considera 33 pizzas distribuídas em diversas categorias, com 124 relações entre pizzas e ingredientes, possibilitando uma base de conhecimento consistente. Com isso, espera-se oferecer uma experiência mais prática e intuitiva na escolha de pizzas, tornando o processo mais rápido, personalizado e eficiente.

2. REPRESENTAÇÃO DO CONHECIMENTO

2.1 ESTRUTURA DOS FATOS

A base de conhecimento do sistema foi organizada utilizando diferentes predicados, cada um responsável por armazenar um tipo específico de informação sobre as pizzas. Essa estrutura permite que o Prolog faça inferências e responda consultas de forma eficiente. A seguir, são apresentados exemplos de como cada tipo de fato foi representado:

O predicado **pizza/4** armazena as informações básicas de cada pizza, incluindo seu nome, categoria, molho base e nível de popularidade:

```
% Pizza: nome, categoria, molho base, popularidade
pizza(calabresa, tradicional, tomate, alta).
pizza(chocolate, doce, chocolate, alta).
```

O predicado **ingrediente/2** relaciona cada pizza aos seus ingredientes, permitindo consultas sobre composição:

```
% Ingredientes de cada pizza
ingrediente(calabresa, calabresa).
```

```
ingrediente(calabresa, mussarela).  
ingrediente(calabresa, cebola).
```

O predicado **caracteristica/2** associa características especiais às pizzas, como ser vegetariana ou picante:

```
% Caracteristicas  
caracteristica(margherita, vegetariana).  
caracteristica(calabresa, picante).
```

O predicado **harmoniza_bebida/2** indica quais bebidas combinam melhor com cada pizza:

```
% Harmonizacao com bebidas  
harmoniza_bebida(calabresa, cerveja_pilsen).  
harmoniza_bebida(chocolate, leite).
```

O predicado **ocasio/2** sugere em quais momentos cada pizza é mais indicada:

```
% Ocasioes  
ocasio(calabresa, festa).  
ocasio(chocolate, sobremesa).
```

O predicado **preco/2** classifica as pizzas por faixa de preço:

```
% Faixa de preco  
preco(calabresa, barata).  
preco(camarao, cara).
```

2.2 CONTAGEM DOS FATOS

A tabela abaixo apresenta a quantidade de fatos cadastrados para cada predicado, totalizando 237 fatos na base de conhecimento:

Predicado	Quantidade	Descrição
pizza/4	33	Pizzas cadastradas
ingrediente/2	124	Relacoes pizza-ingrediente
caracteristica/2	29	Vegetariana, picante, etc.
harmoniza_bebida/2	20	Harmonizacao com bebidas
ocasio/2	18	Ocasioes recomendadas
preco/2	13	Faixa de preco
Total	237	Fatos na base

3. REGRAS IMPLEMENTADAS

3.1 REGRAS BÁSICAS (COM CORTE)

As regras básicas são responsáveis por verificações simples e consultas diretas à base de conhecimento. O operador de corte (!) é utilizado para evitar buscas desnecessárias após encontrar uma resposta válida, melhorando a eficiência do sistema.

A regra **pizza_existe/1** verifica se uma pizza está cadastrada no sistema. O corte garante que, ao encontrar a pizza, o Prolog não continue procurando outras ocorrências:

```
%% pizza_existe(+Nome) - COM CORTE
pizza_existe(Nome) :-
    pizza(Nome, _, _, _), !.
```

A regra **pizza_popular/1** busca pizzas populares, priorizando as de alta popularidade. Se não encontrar nenhuma com popularidade alta, busca as de média:

```
%% pizza_popular(-Nome) - COM CORTE
pizza_popular(Nome) :-
    pizza(Nome, _, _, alta), !.
pizza_popular(Nome) :-
    pizza(Nome, _, _, media).
```

3.2 REGRAS COM LISTAS

O Prolog oferece o predicado **findall/3** para coletar todos os resultados de uma consulta em uma lista. Essas regras são essenciais para retornar conjuntos de informações ao usuário.

A regra **ingredientes_pizza/2** coleta todos os ingredientes de uma pizza específica em uma lista:

```
%% ingredientes_pizza(+Pizza, -Lista)
ingredientes_pizza(Pizza, Lista) :-
    pizza_existe(Pizza),
    findall(Ing, ingrediente(Pizza, Ing), Lista).
```

A regra **pizzas_com_ingrediente/2** faz o caminho inverso, listando todas as pizzas que contêm um determinado ingrediente:

```
%% pizzas_com_ingrediente(+Ingrediente, -Lista)
pizzas_com_ingrediente(Ingrediente, Lista) :-
    findall(Pizza, ingrediente(Pizza, Ingrediente), Lista).
```

3.3 REGRAS RECURSIVAS

A recursividade é um conceito fundamental em Prolog, permitindo processar estruturas de dados como listas de forma elegante. Cada regra recursiva possui um caso base (condição de parada) e um caso recursivo (que processa o restante da estrutura).

A regra **membro/2** verifica se um elemento pertence a uma lista, percorrendo-a recursivamente:

```
%% membro(+Elem, +Lista) - RECURSIVA
membro(X, [X|_]) :- !.
membro(X, [_|T]) :- membro(X, T).
```

A regra **tamanho/2** conta quantos elementos uma lista possui, somando 1 a cada chamada recursiva:

```
%% tamanho(+Lista, -N) - RECURSIVA
tamanho([], 0).
tamanho(_|T, N) :-
    tamanho(T, N1),
    N is N1 + 1.
```

A regra **filtrar_vegetarianas/2** percorre uma lista de pizzas e retorna apenas aquelas que possuem a característica vegetariana:

```
%% filtrar_vegetarianas(+Lista, -Filtrada) - RECURSIVA
filtrar_vegetarianas([], []).
filtrar_vegetarianas([Pizza|Resto], [Pizza|Filtrada]) :-
    caracteristica(Pizza, vegetariana), !,
    filtrar_vegetarianas(Resto, Filtrada).
filtrar_vegetarianas(_|Resto, Filtrada) :-
    filtrar_vegetarianas(Resto, Filtrada).
```

3.4 REGRAS DE RECOMENDAÇÃO

As regras de recomendação representam a inteligência do sistema, combinando diferentes critérios para sugerir pizzas adequadas a cada perfil de usuário. Elas demonstram como o Prolog pode fazer inferências a partir da base de conhecimento.

A regra **recomendar_para_vegetariano/2** sugere pizzas vegetarianas que não sejam doces, retornando também o motivo da recomendação:

```
%% recomendar_para_vegetariano(-Pizza, -Motivo)
recomendar_para_vegetariano(Pizza, Motivo) :-
    caracteristica(Pizza, vegetariana),
    pizza(Pizza, Cat, _, _),
    Cat \= doce,
    Motivo = 'Opcao vegetariana saborosa', !.
```

A regra **recomendar_para_festa/2** sugere pizzas tradicionais de alta popularidade, ideais para agradar a maioria dos convidados:

```
%% recomendar_para_festa(-Pizza, -Motivo)
recomendar_para_festa(Pizza, Motivo) :-
    pizza(Pizza, tradicional, _, alta),
```

```
Motivo = 'Popular, agrada a maioria'.
```

3.5 CONTAGEM DE REGRAS

A tabela abaixo apresenta a distribuição das 42 regras implementadas por categoria:

Categoria	Quantidade
Regras básicas	3
Busca por características	5
Operacoes com listas	4
Regras recursivas	8
Recomendacoes	6
Consultas avançadas	3
Interface	13
Total	42

4. USO DE CORTES

O operador de corte (!) é uma ferramenta importante em Prolog para controlar o fluxo de execução e evitar backtracking desnecessário. Seu uso adequado melhora a eficiência do programa e evita resultados duplicados.

4.1 EM VERIFICAÇÕES

Nas verificações de existência, o corte impede que o Prolog continue buscando após encontrar a primeira ocorrência:

```
pizza_existe(Nome) :-  
    pizza(Nome, _, _, _), !.
```

4.2 EM BUSCAS COM PRIORIDADE

Em regras com múltiplas cláusulas, o corte estabelece uma ordem de prioridade. No exemplo abaixo, pizzas de alta popularidade são retornadas primeiro:

```
pizza_popular(Nome) :-  
    pizza(Nome, _, _, alta), !. % Prioriza alta popularidade  
pizza_popular(Nome) :-  
    pizza(Nome, _, _, media).
```

4.3 EM FILTROS RECURSIVOS

Nos filtros recursivos, o corte evita que elementos já processados sejam reavaliados:

```
filtrar_vegetarianas([Pizza|Resto], [Pizza|Filtrada]) :-  
    caracteristica(Pizza, vegetariana), !,  
    filtrar_vegetarianas(Resto, Filtrada).
```

4.4 NO MENU

No processamento do menu, o corte garante que apenas uma opção seja executada por vez, evitando comportamentos inesperados:

```
processar(0) :- !, write('Ate logo!'), nl.  
processar(1) :- !, buscar_nome, menu_principal.
```

5. USO DE RECURSIVIDADE

A recursividade é a técnica de definir uma regra em termos de si mesma, sendo fundamental para processar listas em Prolog. Toda regra recursiva precisa de um caso base para evitar loops infinitos.

5.1 OPERAÇÕES COM LISTAS

A regra **tamanho/2** demonstra uma recursão clássica: o caso base retorna 0 para lista vazia, e o caso recursivo soma 1 ao tamanho do restante da lista:

```
tamanho([], 0).  
tamanho([_|T], N) :-  
    tamanho(T, N1),  
    N is N1 + 1.
```

A regra **concatenar/3** junta duas listas, transferindo elemento por elemento da primeira lista para o resultado:

```
concatenar([], L, L).  
concatenar([H|T], L2, [H|L3]) :-  
    concatenar(T, L2, L3).
```

5.2 FILTROS

A regra **filtrar_por_categoria/3** percorre uma lista de pizzas e mantém apenas aquelas que pertencem a uma categoria específica:

```
filtrar_por_categoria([], _, []).  
filtrar_por_categoria([Pizza|Resto], Cat, [Pizza|Filtrada]) :-  
    pizza(Pizza, Cat, _, _), !,  
    filtrar_por_categoria(Resto, Cat, Filtrada).  
filtrar_por_categoria([_|Resto], Cat, Filtrada) :-
```

```
filtrar_por_categoria(Resto, Cat, Filtrada).
```

5.3 EXIBIÇÃO

A regra **exibir_numerada/2** mostra uma lista com numeração sequencial, incrementando o contador a cada chamada recursiva:

```
exibir_numerada([], _).  
exibir_numerada([H|T], N) :-  
    format(' ~w. ~w~n', [N, H]),  
    N1 is N + 1,  
    exibir_numerada(T, N1).
```

6. USO DE LISTAS

Listas são estruturas de dados fundamentais em Prolog, permitindo armazenar e manipular coleções de elementos. O sistema utiliza listas tanto para coletar resultados quanto para filtrá-los.

6.1 COLETA DE FINDALL

O predicado **findall/3** coleta todos os resultados que satisfazem uma condição em uma lista. É utilizado para reunir pizzas de uma categoria ou ingredientes de uma pizza:

```
pizzas_categoria(Cat, Lista) :-  
    findall(Nome, pizza(Nome, Cat, _, _), Lista).  
  
ingredientes_pizza(Pizza, Lista) :-  
    findall(Ing, ingrediente(Pizza, Ing), Lista).
```

6.2 FILTRAGEM

As regras de filtragem processam listas existentes, selecionando apenas os elementos que atendem a critérios específicos:

```
filtrar_vegetarianas(ListaOriginal, ListaFiltrada).  
filtrar_por_categoria(ListaOriginal, Categoria, ListaFiltrada).
```

7. INTERFACE

7.1 MENU PRINCIPAL

A interface do sistema foi projetada para ser simples e intuitiva, permitindo que o usuário navegue pelas funcionalidades através de um menu numerado:

```

*****
*                               MENU PRINCIPAL                               *
*****
*  1. Buscar pizza por nome                                             *
*  2. Listar por categoria                                              *
*  3. Buscar por ingrediente                                            *
*  4. Recomendacoes                                                    *
*  5. Harmonizacao com bebidas                                         *
*  6. Comparar pizzas                                                  *
*  7. Estatisticas                                                      *
*  0. Sair                                                              *
*****

```

7.2 FUNCIONALIDADES

Cada opção do menu oferece uma funcionalidade específica para auxiliar o usuário na escolha da pizza ideal:

Opção	Descrição
1	Info completa de uma pizza
2	Lista por categoria
3	Busca por ingrediente
4	Recomendacoes personalizadas
5	Harmonizacao com bebidas
6	Comparar duas pizzas
7	Estatísticas gerais

8. EXEMPLO DE USO

8.1 INFORMAÇÃO DE UMA PIZZA

Ao consultar uma pizza específica, o sistema exibe todas as informações disponíveis sobre ela:

```

?- info_completa(calabresa).

=====
PIZZA: calabresa
=====
Categoria: tradicional
Molho base: tomate
Popularidade: alta
Ingredientes: [calabresa,mussarela,cebola]

```



```
Características: [picante]
Harmoniza com: [cerveja_pilsen, refrigerante]
Faixa de preço: barata
=====
```

8.2 BUSCAR POR INGREDIENTE

É possível descobrir quais pizzas contêm um determinado ingrediente:

```
?- pizzas_com_ingrediente(mussarela, Lista).
Lista = [margherita, mussarela, calabresa, portuguesa, ...]
```

8.3 RECOMENDAÇÃO

O sistema oferece recomendações contextualizadas, indicando pizzas adequadas para cada ocasião:

```
?- recomendar_para_festa(P, M).
P = margherita, M = 'Popular, agrada a maioria' ;
P = calabresa, M = 'Popular, agrada a maioria'
```

9. CONCLUSÃO

O desenvolvimento deste Sistema Especialista em Prolog permitiu explorar de forma prática os principais conceitos do paradigma lógico de programação. Ao longo do projeto, foi possível compreender como representar conhecimento através de fatos e regras, e como o mecanismo de inferência do Prolog utiliza unificação e backtracking para responder consultas de forma automática.

A escolha do tema "Guia de Pizzas" mostrou-se adequada para demonstrar as capacidades da linguagem, uma vez que permitiu criar uma base de conhecimento rica e diversificada, com 237 fatos distribuídos em diferentes predicados. Essa estrutura possibilitou a implementação de consultas variadas, desde buscas simples até recomendações personalizadas que combinam múltiplos critérios.

O uso do operador de corte foi essencial para otimizar o desempenho do sistema, evitando buscas redundantes e garantindo que as respostas fossem retornadas de forma eficiente. Já a recursividade demonstrou ser uma ferramenta poderosa para manipular listas, permitindo implementar filtros, contadores e funções de exibição de maneira elegante e concisa.

A interface interativa desenvolvida facilita o uso do sistema por usuários sem conhecimento técnico em Prolog, abstraindo a complexidade das consultas através de menus intuitivos. As funcionalidades implementadas, como busca por ingredientes, recomendações por perfil e harmonização com bebidas, agregam valor prático ao sistema e demonstram como a programação lógica pode ser aplicada em situações do cotidiano.

10. APÊNDICE: PIZZAS DISPONÍVEIS

Tradicionais: margherita, mussarela, calabresa, portuguesa, frango_catupiry, quatro_queijos, napolitana, romana, siciliana, pepperoni

Especiais: camarao, salmao, file_mignon, lombo_canadense, bacon_cheddar, costela, pulled_pork, buffalo_chicken

Veganas: vegetariana, rucula_tomate_seco, cogumelos, abobrinha, berinjela, caprese

Doces: chocolate, brigadeiro, romeu_julieta, banana_canela, morango_nutella, prestigio

Fitness: low_carb, integral, proteica